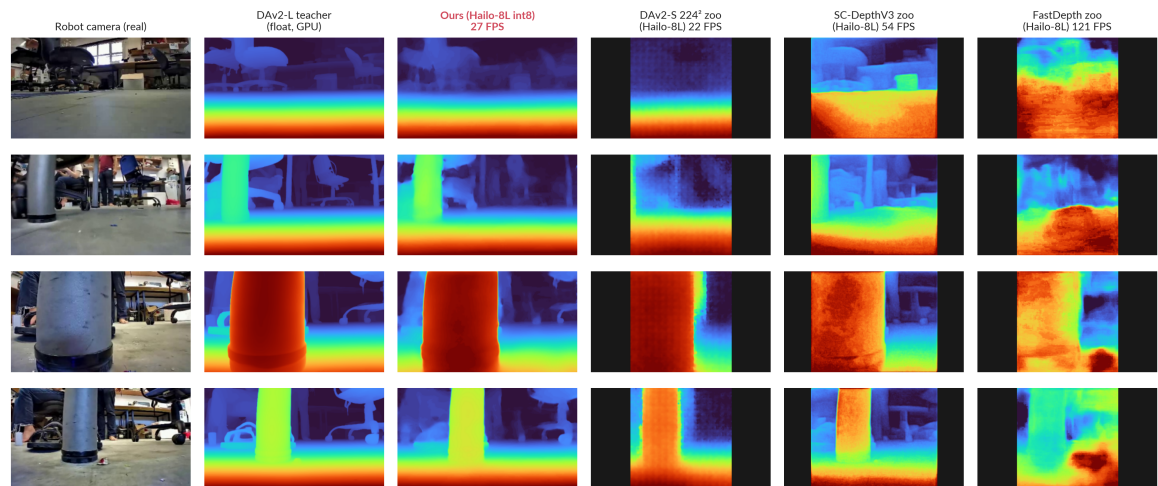

Seeing to Drive on a Single Board

A distilled, attention-free monocular depth network on a Raspberry Pi 5 + Hailo-8L, and the onboard autonomy it enables: point clouds, obstacle avoidance, global path planning and SLAM for a small rover

Itai M.

Thesis project DANIEL – Depth Anything Navigation In Exteriors, Learned

Real robot camera, depth computed on the Raspberry Pi 5 + Hailo-8L (dark = outside a model's input crop)



Version of 2 October 2026. Every number in this document is a measurement from the project's own experiments; the scripts that produced each table and figure are listed in Appendix A.

Small ground robots rarely carry LiDAR or stereo; a single camera and a few watts of compute are what they can afford. Monocular depth foundation models such as Depth Anything V2 now produce remarkably good relative depth, but they are vision transformers sized for GPUs. This thesis takes one of them to a Raspberry Pi 5 with a 13-TOPS Hailo-8L neural accelerator and builds a complete onboard autonomy stack on top of it.

We first show, by measurement on the chip, *why* transformer depth breaks on this accelerator: the Hailo-8L executes attention in int8, and at 449 tokens a diffuse attention weight falls below a single quantisation step. Our own int8 build of Depth Anything V2-Small, and Hailo’s official one, lose a large part of the scene. We therefore distil Depth Anything V2-Large into an *attention-free* ResNet-34 U-Net (24.5 M parameters) and fine-tune it on rendered ground truth. On the real Hailo-8L it runs at **27.1 FPS at 288×512**, its int8 output matches its own float twin at Pearson **0.99995**, and on 400 held-out frames with metric ground truth it is the most accurate of every depth network we ran on the chip (AbsRel 0.269 overall versus 0.354–0.530 for every other build, including the vendor’s), while agreeing with its 335 M-parameter teacher at $r = 0.995$ on real robot video.

Relative depth has no metric scale, so we range obstacles by *ground-contact geometry* instead: the image row where an object meets the floor fixes its distance from the camera height alone. On this scan we build a hit-count map, hybrid-A* global planning, pure-pursuit control, and a SLAM front end (ground-plane visual odometry, optional wheel odometry, and an image-based slip guard). The whole pipeline fits the Pi: 35 ms on the NPU in parallel with ≈ 47 ms on the CPU (median, deployed code), with planning in an asynchronous worker.

We evaluate the system with the real Pi and Hailo-8L in a real-time hardware-in-the-loop simulation: a Gaussian-splat reconstruction of a real yard in Isaac Sim, eleven kinds of realistic obstacles with honest colliders, ten scenes and at least five seeds per configuration, scored on the car’s real footprint. The deployed stack reaches the goal without any contact in **7.9 ± 1.1 of 10** scenes with simulator pose and **7.7 ± 0.7** with its own SLAM pose (simulated wheel encoder and gyro; ten seeds each), **8.2 ± 0.4** with the real board in the loop, against **9.2** for the same real-time stack fed exact obstacles and 9.8 for an oracle. Camera-only odometry reaches 2.6. We report the full error budget, more than a dozen negative results, and what remains between this system and a perfect score: obstacle bases lower than the scan’s 5 cm threshold, and exploration around obstacles the camera has not yet seen.

Contents

Abstract	2
1 Introduction	5
1.1 The problem in one paragraph	5
1.2 Contributions	6
1.3 Headline numbers	7
1.4 Outline	7
2 Background and related work	8
2.1 Monocular depth: from regression to foundation models	8
2.2 Depth on small computers	8
2.3 Quantising transformers	9
2.4 Navigating with predicted depth	9
2.5 SLAM	9
2.6 Simulation and hardware in the loop	9
2.7 Positioning	10
3 An attention-free depth student for the Hailo-8L	11
3.1 Platform	11
3.2 Why transformer depth breaks on this chip	11
3.3 The student	12
3.4 Compilation and on-chip fidelity	13
4 Benchmark: depth on the edge	15
4.1 Protocol	15
4.2 Results	16
4.3 What the benchmark does and does not show	17
5 From relative depth to metric space	19
5.1 Ground-contact geometry	19
5.2 The scan	19
5.3 Point clouds	21
5.4 The map	21
5.5 The measured limit: low bases	21
6 Onboard autonomy: planning, control and SLAM	23
6.1 Global planning: hybrid A* on sensed obstacles	23
6.2 Control and safety behaviours	24
6.3 SLAM	24
6.4 The real-time budget on the Pi	25

7	Hardware-in-the-loop evaluation	27
7.1	Set-up	27
7.2	Results	28
7.3	Failure analysis	29
7.4	What did not help	31
7.5	Statistics	32
8	Discussion	33
8.1	What is new	33
8.2	Limits, stated plainly	34
8.3	Threats to validity	34
9	Conclusion and future work	35
9.1	Closing the gap	35
9.2	Before the first autonomous drive	35
	Bibliography	37
A	Reproduction	41
A.1	Depth student	41
A.2	Benchmark (Chapter 4)	41
A.3	Navigation stack and HITL	42
A.4	Commands	42

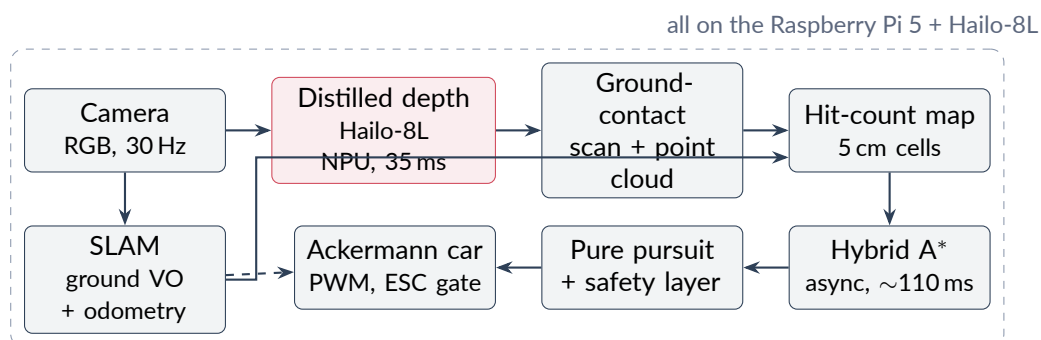
Chapter 1

Introduction

A small rover that has to find its way through a yard full of chairs, cones, crates and people needs to know where things are. The classical answer is a range sensor: a 2D or 3D LiDAR, a stereo pair, or a time-of-flight camera. Each costs money, power, mass and integration effort, and each has blind spots of its own. A planar LiDAR does not see a table top or a low curb; stereo needs texture and a baseline that a palm-sized vehicle does not have. A single RGB camera is the cheapest and lightest sensor a robot can carry, and it sees all of these things. What it does not give directly is *distance*.

In the last three years monocular depth estimation has changed character. Networks trained on tens of millions of images and distilled from synthetic ground truth—MiDaS [28], Depth Anything [42], Depth Anything V2 [43], Depth Pro [1] and, most recently, Depth Anything 3 [18]—now predict relative depth that is sharp, stable and general enough to drive with. They are, however, vision transformers sized for data-centre GPUs: Depth Anything V2-Large has 335 M parameters and, even on the RTX 5090 used in this work, takes 175 ms per frame at its native resolution. A small robot has a few watts.

This thesis asks how much of that capability survives on the cheapest serious edge platform available today, a Raspberry Pi 5 with the 13-TOPS Hailo-8L accelerator of the Raspberry Pi AI Kit [29], and what a rover can do with it. The answer we give is a complete, measured system:



1.1 The problem in one paragraph

Running Depth Anything V2 on the Hailo-8L sounds like a compilation task. It is not. The accelerator executes every layer in integer arithmetic, attention included, and attention over

a few hundred image tokens produces weights so small that int8 cannot represent them. The obvious build loses large parts of the scene, and so does the vendor's own build of the same network (Chapters 3 and 4). Even with a faithful depth map, the output is *relative*: correct up to an unknown scale and shift, so it cannot by itself tell a planner that a chair is 1.2 m away. And even with metric obstacles, a car needs a map, a planner, a controller and an estimate of its own position, all inside the same few watts and the same 50-millisecond control period.

1.2 Contributions

- C1 A measured explanation of why transformer depth fails on the Hailo-8L, and an attention-free student that does not.** We show on the chip that int8 attention erases obstacles in Depth Anything V2-Small builds (ours and the vendor's), and we distil Depth Anything V2-Large into a 24.5 M-parameter ResNet-34 U-Net with no attention at all. Its int8 output on the real Hailo-8L matches its float twin at Pearson 0.99995 (p5 0.99897, 500 frames), and it runs at 27.1 FPS at 288×512 — $1.7 \times$ the pixels of our Depth Anything V2-S build at $1.6 \times$ its speed (Chapter 3).
- C2 The first head-to-head benchmark of every depth network we could run on the Hailo-8L,** on 400 held-out frames with metric ground truth and 100 real robot frames. The student is the most accurate on the chip by a wide margin and stays within a few points of its 335 M-parameter teacher (Chapter 4).
- C3 Scale-free metric ranging from relative depth.** A ground-contact obstacle scan turns the student's relative depth into a metric laser-like scan using only the camera height, with three fixes we found by failure analysis: a horizon-anchored floor model, a scale-free vertical-face detector and a floating-overhang detector (Chapter 5).
- C4 A complete onboard autonomy stack within the Pi's budget:** point clouds, a hit-count map, hybrid-A* global planning with a safety cascade, pure-pursuit control and a SLAM front end with an encoder-gated slip guard. All of it is measured on the robot computer: NPU 35 ms in parallel with about 49 ms of CPU (Chapter 6).
- C5 A real-time hardware-in-the-loop evaluation with an explicit error budget.** The real Pi and Hailo-8L drive a simulated Traxxas through a Gaussian-splat reconstruction of a real yard. There are ten scenes, eleven obstacle types with honest colliders, at least five seeds per configuration, and contacts are scored on the car's real footprint. Oracle and exact-obstacle ceilings separate perception, planning and localisation error (Chapter 7).
- C6 An honest record.** More than a dozen interventions that did *not* help are reported with the same rigour as those that did, together with the measured limits that remain (Chapter 8).

1.3 Headline numbers

On the Raspberry Pi 5 + Hailo-8L: 27.1 FPS depth at 288×512 , int8 = float ($r = 0.99995$), AbsRel 0.269 on held-out metric GT (next best on the chip: 0.354). In the real-time yard suite: 7.9 ± 1.1 of 10 scenes reached without contact with simulator pose and 7.7 ± 0.7 on the robot's own SLAM pose (ten seeds each), 8.2 ± 0.4 with the real board in the loop, against 9.2 for the same stack fed exact obstacles.

1.4 Outline

Chapter 2 places the work among recent monocular depth, edge-inference and depth-based navigation research. Chapter 3 explains the int8 attention failure and the student that avoids it. Chapter 4 benchmarks it against every alternative on the chip and against the float foundation models. Chapter 5 turns relative depth into metric space. Chapter 6 describes the planner, controller and SLAM and their real-time budget on the Pi. Chapter 7 evaluates the whole system in the loop. Chapter 8 discusses what is new, what is not, and what remains. Chapter 9 concludes and lists what the first real drive needs.

Chapter 2

Background and related work

2.1 Monocular depth: from regression to foundation models

Learning depth from one image goes back to multi-scale regression on indoor and driving datasets [7]. Two ideas made it general. MiDaS [28] trained on a mixture of datasets with a *scale- and shift-invariant* loss, predicting *relative* inverse depth that transfers zero-shot. DPT [27] replaced the convolutional encoder with a vision transformer. Depth Anything [42] scaled this recipe to 62 M unlabelled images through pseudo-labelling, and Depth Anything V2 [43] trained a large teacher on purely synthetic, precise ground truth and distilled it through pseudo-labelled real images into students of 25 M (Small), 98 M (Base) and 335 M (Large) parameters. Its detail and robustness are why it is our teacher. A parallel line predicts *metric* depth directly, by conditioning on or estimating the camera: Metric3D v2 [12], UniDepth [26] and Depth Pro [1], which produces a 2.25-megapixel metric map in 0.3 s on a GPU. Generative models repurposed for depth, such as Marigold [15], trade speed for detail. Most recently, Depth Anything 3 [18] uses a single plain transformer with a depth-ray target to recover consistent geometry from any number of views; Video Depth Anything [4] adds temporal consistency to Depth Anything V2; and Distill Any Depth [10] shows that distillation choices alone, notably avoiding global normalisation of noisy pseudo-labels, produce a stronger estimator. A 2026 survey [20] traces this whole arc. Every one of these models is a transformer, and none was designed for an integer-only accelerator.

2.2 Depth on small computers

FastDepth [39] showed real-time depth on a Jetson TX2 with a pruned MobileNet encoder-decoder (over 120 FPS on its GPU). Self-supervised lightweight models such as Lite-Mono [45] and SC-DepthV3 [36] learn from video without labels. FastDepth and SC-DepthV3 are the two depth models Hailo ships in its Model Zoo for the Hailo-8L, and both appear in our benchmark. AsyncMDE [22] amortises a foundation model over time: a 97.5 M frozen slow path writes spatial memory, and a 3.8 M fast path runs at 161 FPS on a Jetson AGX Orin. Closest in spirit to our platform, Jalalnezhad et al. [14] run a lightweight monocular depth network at 14–20 Hz on embedded CPUs (Raspberry Pi 4 class) with behaviour-based control. Our work differs in three ways. The accelerator is an integer-only NPU rather than a GPU or a CPU. The network is a student of a foundation model rather

than a model trained from scratch. And the depth feeds a full mapping, planning and SLAM stack rather than a reactive behaviour.

2.3 Quantising transformers

Integer inference [13] is what makes NPUs efficient. Vision transformers are known to be hard to quantise because post-softmax and post-LayerNorm activations have long-tailed, power-law distributions. FQ-ViT [19], PTQ4ViT [44] and RepQ-ViT [17] address this with log-domain softmax quantisers, twin uniform ranges and scale reparameterisation, each of which needs support in the target hardware’s quantiser. On the Hailo-8L, attention is executed as int8 with a uniform quantiser that we cannot change. Chapter 3 measures what that does to Depth Anything V2, and explains why, for this chip, removing attention through knowledge distillation [11] is more practical than better quantisation.

2.4 Navigating with predicted depth

MonoNav [34] fuses pre-trained transformer depth into a metric reconstruction and plans through it, flying a 37 g Crazyflie at 0.5 m/s, with depth computed off-board. MonoMPC [33] uses estimated depth not for direct collision checking but as context for a learned collision model inside a risk-aware MPC. MonoSpheres [24] explores large outdoor environments from a sparse monocular SLAM front end. Finke et al. [8] is the closest recent work. They fine-tune Depth Anything V2 for four onboard cameras, train a reinforcement-learning student policy in Isaac Lab, and run everything on a Jetson AGX Orin at 10 Hz, outperforming a 2D LiDAR on overhangs and low objects. Learned navigation policies such as ViNT [32] and NoMaD [35], and imperative planners such as iPlanner [41] and ViPlanner [31], map images or depth directly to paths. We deliberately keep the planning classical—hybrid A^* [6], pure pursuit [5], and a reactive arc scorer in the tradition of the vector field histogram [2]—so that every failure can be attributed to perception, planning or localisation.

2.5 SLAM

ORB-SLAM3 [3] is the reference feature-based system. DROID-SLAM [37], DPVO [38] and MAST3R-SLAM [23] (15 FPS with GPU kernels) are dense or learned alternatives that need a GPU. On a Pi CPU shared with planning we use a deliberately small front end: ground-plane visual odometry that lifts tracked floor features [21] to metric points with the known camera height, optionally fused with wheel odometry and a gyro, plus a rotation-compensated image-stillness test that detects wheel slip (Chapter 6).

2.6 Simulation and hardware in the loop

We evaluate in NVIDIA Isaac Sim with Isaac Lab [25]. The yard is a Gaussian-splat reconstruction [16] of a real yard, rendered through NVIDIA’s neural-reconstruction path, which builds on the 3DGUT formulation [40]. The robot computer is not simulated. The

Table 2.1. Closest systems that navigate with predicted monocular depth.

	Depth model and where it runs	Navigation	Evaluation
MonoNav [34]	pre-trained transformer, <i>off-board</i>	metric fusion + motion primitives	real indoor MAV flights
Finke et al. [8]	fine-tuned Depth Anything V2, 4 cameras, Jetson AGX Orin, 10 Hz	RL policy (teacher-student)	Isaac Lab + real robot
MonoMPC [33]	estimated depth as context	learned collision model + MPC	sim + real
Jalalnezhad et al. [14]	lightweight CNN, embedded CPU, 14–20 Hz	behaviour-based control	real indoor/outdoor
This thesis	distilled attention-free student, Hailo-8L int8, 27 FPS	ground-contact scan, hybrid A*, SLAM, all on the Pi	real-time HITL with the real board, ≥ 5 seeds, error budget

real Raspberry Pi 5 and Hailo-8L receive every camera frame over the network and send back commands, which land after the measured round trip.

2.7 Positioning

Table 2.1 contrasts the closest systems. To our knowledge this is the first work that (i) runs a foundation-model-quality depth network on a 13-TOPS, int8-only Raspberry Pi accelerator, (ii) builds metric mapping, global planning and SLAM on it within the Pi’s own budget, and (iii) evaluates the whole loop with the real board in a reconstructed real environment, with an explicit decomposition of the remaining error.

Chapter 3

An attention-free depth student for the Hailo-8L

3.1 Platform

The robot computer is a Raspberry Pi 5 (four Cortex-A76 cores) with the Raspberry Pi AI Kit: a Hailo-8L module on an M.2 HAT, rated at 13 TOPS and about 1.5 W typical [29] (the module has no power sensor that HailoRT can read, so this thesis reports no measured power). Networks are compiled offline with Hailo’s Dataflow Compiler into a HEF file and executed through HailoRT. The Hailo-8L is an integer accelerator: on this part every layer runs with 8-bit activations and weights, including the matrix products and softmax of attention.

3.2 Why transformer depth breaks on this chip

Our first step was the obvious one: export Depth Anything V2-Small [43] at 224×392 (a 16×28 grid of 14-pixel patches plus a class token, 449 tokens), calibrate it on domain images, and compile. On real robot video the result looked right (median Pearson 0.99 against the float model). On the yard renders it did not: on a third of held-out frames it *erased* obstacles, producing depth maps in which a crate or a person dissolves into the floor (Figure 4.3, fourth column).

We traced this to attention in int8 (Figure 3.1). A transformer head whose attention is spread over many tokens produces softmax weights of order $1/449 \approx 0.0022$. A uniform 8-bit quantiser over $[0, 1]$ has a step of $1/255 \approx 0.0039$. A diffuse head is therefore rounded to zero or to one step, and what it was averaging is lost. This is exactly the long-tail problem that ViT quantisation methods attack with log-domain or twin-range softmax quantisers [17, 19, 44]. The Hailo-8L toolchain applies its own int8 scheme to these layers (a8_w8_a8), and we found no supported way to give softmax a different quantiser. We tried the remedies that were available:

- re-calibration on domain images plus the compiler’s quantisation fine-tuning moved the share of held-out frames with Pearson < 0.8 only from 48 % to 37 %;
- sensor-noise and sharpening camera models in calibration left 30–39 % of frames broken;

Depth Anything V2-S at the Hailo-8L's input: 449 tokens, softmax weights measured in float on 26 frames

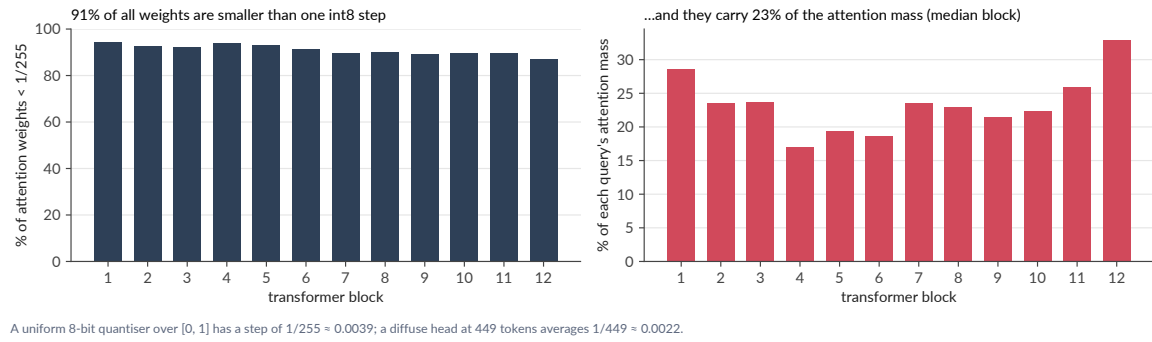


Figure 3.1. Measured in float Depth Anything V2-Small at the Hailo-8L's 224×392 input (449 tokens) on 26 held-out sim and real frames: 91 % of all softmax weights are smaller than one step of a uniform 8-bit quantiser over $[0, 1]$, and they carry 17–33 % of each query's attention mass (median 23 %; most in the last block, which feeds the depth head).

- Hailo's own Model Zoo build of Depth Anything V2-Small (224×224) shows the same blockiness and object loss (Figure 4.3, fifth column; Chapter 4).

We measured how much of the computation lives below that step (Figure 3.1). Across the twelve blocks, 91 % of the weights are below one int8 step, and they carry 17–33 % of every query's attention mass. That is the mass an 8-bit uniform quantiser rounds to zero or to a single step.

The failure is structural: int8 cannot represent diffuse attention at a few hundred tokens. Better calibration does not fix it; removing attention does.

3.3 The student

We therefore distil Depth Anything V2-Large (335 M parameters) into a network with *no attention at all* (Figure 3.2). A ResNet-34 encoder [9] (ImageNet initialisation) feeds a U-Net decoder [30] built only from convolution, batch normalisation, ReLU, $2 \times$ bilinear upsampling and skip concatenation, all of which quantise well. It has 24.5 M parameters, almost exactly Depth Anything V2-Small's 24.8 M, so the comparison isolates architecture from capacity. Input is RGB at 288×512 , the aspect of the robot camera. Output is relative inverse depth (disparity) at full input resolution.

Loss. The student is trained with the scale- and shift-invariant L1 loss of MiDaS [28] plus a four-scale gradient-matching term. Both prediction and target are normalised per image by median and mean absolute deviation, so only the *shape* of the depth map is supervised. This is the right target for a relative teacher and, as Chapter 5 shows, all the navigation stack needs.

Data. The teacher labels three sources:

- 7,230 simulated point-of-view frames from hardware-in-the-loop runs of scenes 00–06 (every 0.2 s);

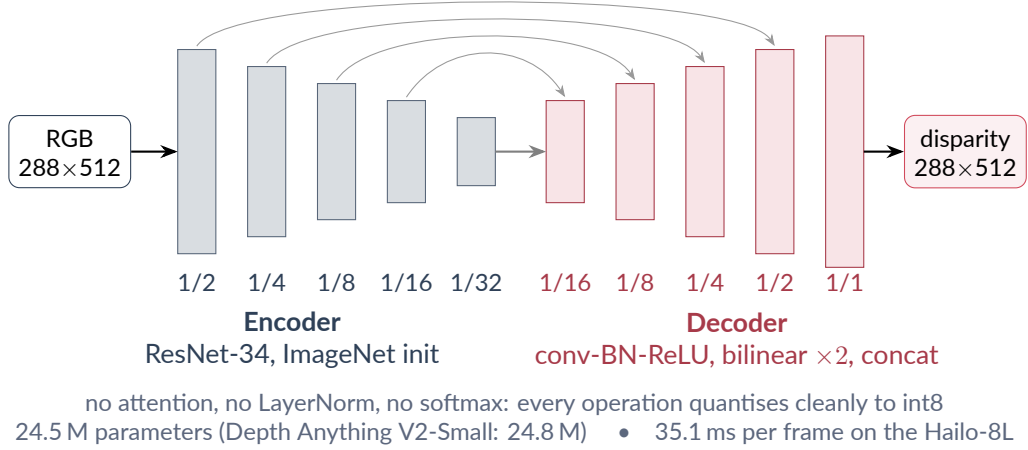


Figure 3.2. The attention-free student. Grey arrows are U-Net skip connections.

- 720 real frames from the robot camera and calibration sets;
- later, 2,958 random-layout renders and 1,304 frames from 36 local videos.

Every frame of scenes 07–09 (crates, people, barrels) and the real robot video are held out completely.

Stages.

v1 distilled from Depth Anything V2-Large on the first set. Held-out Pearson to the teacher: 0.990 (sim) and 0.998 (real video).

v2 the same with the extra data, 20 epochs. It did *not* improve the novel-object scene (crates: frames below $r = 0.8$ rose from 14.5 % to 21.0 %). Closing a novel-object gap needs a far larger and more varied corpus.

v3 v1 fine-tuned for 8 epochs on *rendered* ground-truth disparity for the simulated frames, with the teacher kept for real frames (13,902 simulated + 2,024 real training frames, AdamW, one-cycle learning rate 3×10^{-4} , batch 16, about 150 s per epoch on an RTX 5090). The teacher’s own relative depth had made v1 range simulated props about 30 % short; v3 brings that bias to about -7 %.

v5 v3 with the loss weighted $\times 4$ on the image rows below the horizon (floor and object bottoms). It reduced spurious near-floor hits by a quarter but did not resolve obstacle bases better (Section 5.5). In the loop it ties v3 with simulator pose and is ahead, though not significantly, on the robot’s own pose (Chapter 7). It is the deployed student.

3.4 Compilation and on-chip fidelity

The student compiles with normalisation folded into the HEF and optimisation level 1, calibrated on domain frames. One finding matters for anyone repeating this: the compiler’s own emulator check reported a correlation of -0.41 between int8 and native for this graph,

a red flag that turned out to be false. Replaying the HEF on the real chip against the float model gives:

Student v3, Hailo-8L int8 vs the same weights in float, 500 frames (400 held-out sim + 100 real): Pearson median 0.99995, 5th percentile 0.99897. The quantised network is, for every practical purpose, the float network. Judge an NPU build by replay on the chip, not by the compiler's estimate.

The deployed v5, compiled with the same recipe, replays at Pearson 0.99993 (5th percentile 0.99569) on the same frames.

On the Pi the student takes 35.1 ms on the NPU, or 37.0 ms per frame including pre- and post-processing: 27.1 FPS at batch 1 and 59 FPS at batch 8. Our int8 Depth Anything V2-Small build takes 57.7 ms (16.6 FPS) for fewer pixels. The student therefore delivers $1.7\times$ the pixels at $1.6\times$ the speed, with no visible quantisation loss.

Chapter 4

Benchmark: depth on the edge

4.1 Protocol

Frames. 400 simulated point-of-view frames from 36 hardware-in-the-loop episodes of the three held-out scenes (07 crates, 08 people and cones, 09 barrels), each with the renderer’s metric depth, plus the 100 held-out frames of real robot video, which have no ground truth.

Models. Every depth network we could run on the Hailo-8L: our students (v5, deployed; v3 and v1 for reference), our int8 Depth Anything V2-Small build, and Hailo’s Model Zoo builds of Depth Anything V2-Small (224×224), SC-DepthV3 [36] and FastDepth [39]. All of them run *on the chip*, with the outputs read back. As float references on the GPU we use Depth Anything V2 S/B/L [43] and four Depth Anything 3 variants [18]. These cannot run on the Pi at a useful rate.

Alignment and metrics. Each model sees a centre crop at its own input aspect, and ground truth is cropped to match. Predictions are converted to disparity. Following the standard relative-depth protocol [28], a per-frame least-squares scale and shift aligns them to ground-truth disparity on valid pixels (0.1–20 m). We report AbsRel and δ_1 (fraction of pixels with $\max(\hat{z}/z, z/\hat{z}) < 1.25$) on two regions. *All* is every valid pixel. *Obstacles* is the set of pixels within 3 m that, back-projected with the car’s ground-truth pose, lie on a prop’s collider. The obstacle region matters because the renderer’s depth buffer has speckle and floaters near the floor (visible in Figure 4.3); restricting to pixels on known props removes that noise and measures what the planner actually reads. On real video the metric is the Pearson correlation with the Depth Anything V2-Large teacher in each model’s own crop.

Speed. Frames per second on the Raspberry Pi 5 + Hailo-8L, batch 1, sequential: CPU pre-processing, NPU, read-back and post-processing, median over 500 frames. For reference, the Pi’s four Cortex-A76 cores alone (onnxruntime, four threads) take 455 ms per frame for the student (2.2 FPS) and 381 ms for Depth Anything V2-Small at 224×392: the Hailo-8L makes the student 12× faster. On the RTX 5090 laptop GPU (idle, batch 1, including each model’s own pre- and post-processing) the float models take 38.8 ms (Depth Anything V2-S), 69.9 ms (Depth Anything V2-B), 175.3 ms (Depth Anything V2-L) and about 40 ms (Depth Anything 3 variants); the student takes 4.0 ms.

Table 4.1. Depth on held-out frames. AbsRel/ δ_1 on 400 simulated frames with metric ground truth; “Real r ” is the median Pearson to Depth Anything V2-Large on 100 real robot frames; FPS measured on the Raspberry Pi 5 + Hailo-8L (batch 1). Bold: best on the chip.

Model	Runs on	Input	Params (M)	AbsRel ↓		δ_1 ↑		Real r	FPS
				all	obstacles	all	obstacles	vs DAv2-L	Pi+Hailo
Ours: distilled student v3	Hailo-8L int8	288×512	24.5	0.269	0.204	0.674	0.776	0.995	27.1
Ours: student v5 (floor-weighted)	Hailo-8L int8	288×512	24.5	0.269	0.197	0.669	0.762	0.996	27.1
Ours: student v1 (no sim-GT fine-tune)	Hailo-8L int8	288×512	24.5	0.320	0.188	0.613	0.746	0.996	27.1
DAv2-S, our int8 build	Hailo-8L int8	224×392	24.8	0.477	0.330	0.423	0.591	0.982	16.6
DAv2-S, Hailo Model Zoo	Hailo-8L int8	224×224	24.8	0.354	0.334	0.593	0.619	0.979	21.6
SC-DepthV3, Hailo Model Zoo	Hailo-8L int8	256×320	–	0.414	0.352	0.506	0.688	0.950	53.6
FastDepth, Hailo Model Zoo	Hailo-8L int8	224×224	–	0.530	0.767	0.344	0.477	0.709	121.3
Ours: student v3 (float twin)	GPU fp32	288×512	24.5	0.266	0.200	0.679	0.780	0.995	–
Ours: student v5 (floor-weighted)	GPU fp32	288×512	24.5	0.270	0.200	0.670	0.758	0.996	–
Depth Anything V2-S	GPU fp32	518 short side	24.8	0.283	0.150	0.651	0.813	0.999	–
Depth Anything V2-B	GPU fp32	518 short side	97.5	0.289	0.151	0.637	0.800	0.999	–
Depth Anything V2-L (teacher)	GPU fp32	518 short side	335.3	0.284	0.144	0.643	0.805	1.000	–
Depth Anything 3-Small	GPU fp32	native	34.3	0.315	0.172	0.611	0.760	0.988	–
Depth Anything 3-Base	GPU fp32	native	135.4	0.304	0.188	0.616	0.735	0.985	–
Depth Anything 3-Mono-Large	GPU fp32	native	334.2	0.295	0.171	0.568	0.747	0.983	–
Depth Anything 3-Metric-Large	GPU fp32	native	334.2	0.374	0.239	0.574	0.687	0.988	–

Accuracy vs on-chip speed for every depth network we ran on the Hailo-8L

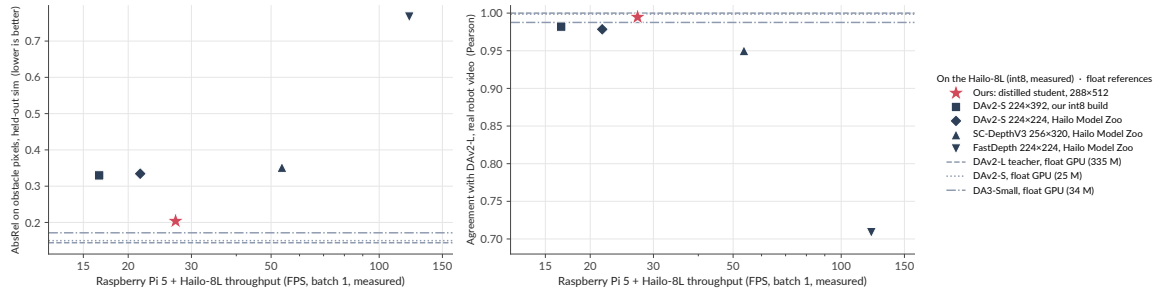


Figure 4.1. Accuracy against measured on-chip throughput. Left: AbsRel on obstacle pixels (lower is better). Right: agreement with Depth Anything V2-Large on real robot video. Horizontal lines are float GPU models that cannot run on the Pi at this rate.

4.2 Results

Table 4.1 and Figure 4.1 summarise the benchmark.

1. **On the chip, the student is in a different class.** Overall AbsRel 0.269 against 0.354 (Hailo’s Depth Anything V2-S), 0.414 (SC-DepthV3), 0.477 (our Depth Anything V2-S build) and 0.530 (FastDepth); δ_1 0.674 against at most 0.593. On obstacle pixels the gap is just as clear: 0.204 against 0.330–0.767. On real video it tracks the teacher at $r = 0.995$, where Hailo’s Depth Anything V2-S reaches 0.979, SC-DepthV3 0.950 and FastDepth 0.709.
2. **Speed is not traded for this.** At 27.1 FPS the student is faster than both Depth Anything V2-S builds on the same chip (21.6 and 16.6 FPS) while processing $2.9\times$ and $1.7\times$ more pixels. Only the two small self-supervised zoo models are faster (54 and 121 FPS), at a large accuracy cost.
3. **Against float foundation models on a GPU, the student gives up a few points on ob-**

stacles but not overall. Depth Anything V2-L, the teacher, reaches 0.144 AbsRel on obstacles against the student's 0.204. Overall the student is *better* (0.266 float, 0.269 on the chip, against 0.283–0.289 for Depth Anything V2 S/B/L), because v3 was fine-tuned on rendered ground truth of this domain. Depth Anything 3 is not better than Depth Anything V2 here (0.17 obstacle AbsRel for the Small and Mono-Large variants).

4. **v1 vs v3.** Fine-tuning on rendered ground truth improves the global metrics (all-pixel AbsRel 0.320→0.269) and the metric range bias, but slightly worsens obstacle pixels (0.188→0.204). In the closed loop the two are tied (Chapter 7). We report the trade rather than pick a favourable metric. v5, the deployed student, matches v3 overall on the chip (0.269) and recovers part of the obstacle loss (0.197).

4.3 What the benchmark does and does not show

The simulated frames come from a reconstruction of the same yard the student was fine-tuned in, although from held-out scenes with objects placed differently and, for crates, never seen in training. The real frames are a different camera in a different building. The student's agreement with its teacher there ($r = 0.995$) shows that sim fine-tuning did not damage real-world behaviour, but agreement with a teacher is not accuracy against ground truth, which we do not have for real frames. Finally, the obstacle region measures depth on props. It does not measure the thin, low structures (cone base plates, chair tubes, feet) that turn out to matter most in the loop; Section 5.5 measures those directly.

Real robot camera, depth computed on the Raspberry Pi 5 + Hailo-8L (dark = outside a model's input crop)

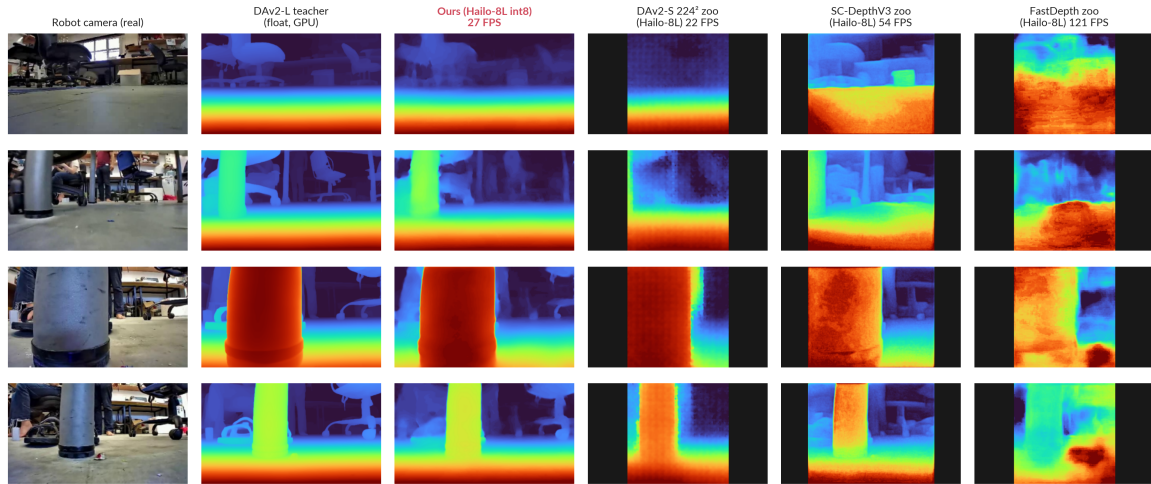


Figure 4.2. Real robot camera. Each column is the depth *as computed on the Hailo-8L* (except the GPU teacher). Dark areas lie outside a model's input crop: the 224×224 Model Zoo builds see only a centre square of the wide camera.

Held-out yard scenes (crates, people, barrels): on-chip depth vs the renderer's depth

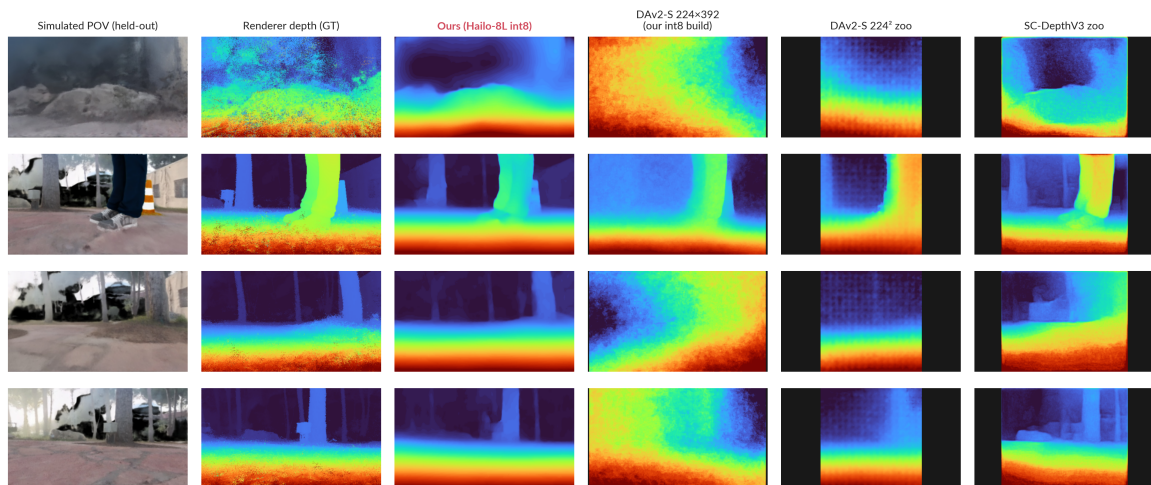


Figure 4.3. Held-out simulated scenes with the renderer's depth (note its speckle near the floor). Our int8 Depth Anything V2-S build scrambles whole frames (int8 attention, Section 3.2); the student stays close to the ground truth.

Chapter 5

From relative depth to metric space

The student predicts disparity up to an unknown scale and shift, $d = a/Z + b$, and its scale drifts from frame to frame. A planner needs metres. Rather than learn metric depth (the route of Metric3D, UniDepth and Depth Pro [1, 12, 26]), we take the metric from geometry the robot already knows: its camera height.

5.1 Ground-contact geometry

For a level camera at height h with focal length f_y and principal row c_y , a floor point imaged at row $v > c_y$ lies at depth

$$Z_{\text{floor}}(v) = \frac{f_y h}{v - c_y}. \quad (5.1)$$

An obstacle standing on the floor meets it at some row v_c ; its range is $Z_{\text{floor}}(v_c)$, whatever the network's scale. The depth network's only job is to say *where in each image column the floor ends*, a question about the *shape* of the depth map, which is exactly what a scale- and shift-invariant loss supervises. The image's columns then become a 69-degree, 4-metre "laser scan" with one range per column (every second column at 288×512).

5.2 The scan

Floor profile. For each row, the 25th percentile of disparity over the central 80 % of columns models the floor (obstacles covering less than about two thirds of a row sit above it). The profile is made monotone (the floor only gets nearer going down the image), limited to rise at most $1.3\times$ between frames, and smoothed in time. The far-field offset b is the 3rd percentile of disparity above the horizon.

Ratio detector. For a pixel at row v , $(d - b)/(f(v) - b) = Z_{\text{floor}}/Z$ is independent of the network's scale. A pixel standing at least $H_{\text{min}} = 5$ cm above the floor at its range has ratio $\geq 1/(1 - H_{\text{min}}/h)$. Scanning each column from the bottom, the first run of five flagged rows is the obstacle's contact, and its range follows from the contact row.

The obstacle scan: metric range from ground-contact geometry, so the depth model's scale never matters

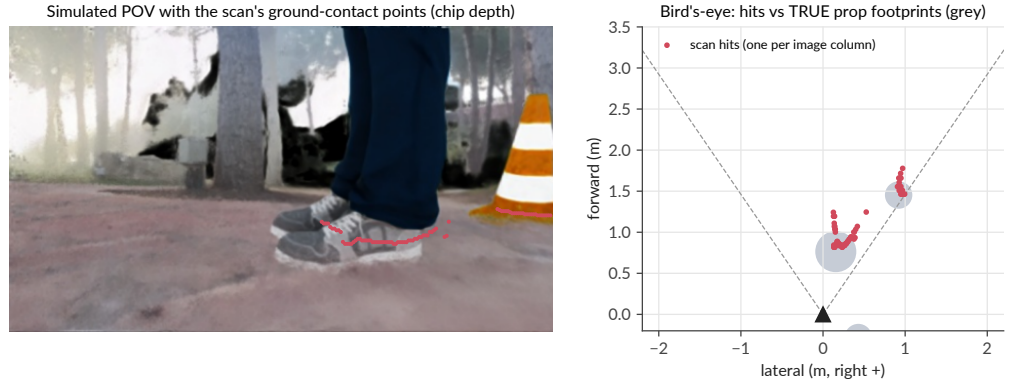


Figure 5.1. The scan on a held-out frame, from depth computed on the Hailo-8L. Left: one ground-contact point per image column. Right: the same hits in bird's-eye view against the *true* prop footprints (the person's collider is a conservative 0.25 m cylinder around the feet).

Vertical-face detector. On the floor, disparity grows linearly down a column, so $s = \frac{\partial d}{\partial v} \frac{v - c_y}{d - b} \approx 1$. On a vertical face disparity is constant, so $s \approx 0$. A run of 16 rows with $s < 0.3$ marks a face. This detector does not use the floor profile, so it still sees a box that fills the whole lower image (which the percentile floor absorbs). Measured slopes are 0.01–0.13 on a box at 0.45 m against 0.77–0.94 on clean floor.

Height. Following the contact disparity upward gives each obstacle's top height. Splat floaters and leaves are a few centimetres tall; legs, cones and boxes are not. Only hits at least 6 cm tall are mapped (except within 0.6 m, where blurred thin legs read short). On six recorded episodes this cut phantom hits from 2,376 to 75 while keeping 56 % of the hits on props.

Two fixes found by failure analysis.

- *Horizon-anchored floor.* The monotone floor profile was originally accumulated from the top of the image. Under a chair, the seat (near, *above* the horizon) was copied into every floor row, so the floor read nearer than the chair's own tubes. Even ground-truth depth then mapped a tube at 0.68 m as 1.16 m. Starting the accumulation at the horizon gives 0.76 m. Together with the planner fixes of Chapter 6, the chair scene went from 0/5 to 3/5 clean.
- *Floating overhangs.* A seat or table top 0.25–0.60 m up with open space under it is mapped as an obstacle at its own range, ranged with the floor profile as a ruler. In the chair and person scenes, 82–91 % of the columns this rule adds are confirmed by the same rule on ground-truth depth (59–76 % around crates and cones).

One camera frame to metric 3D on the Raspberry Pi 5 + Hailo-8L

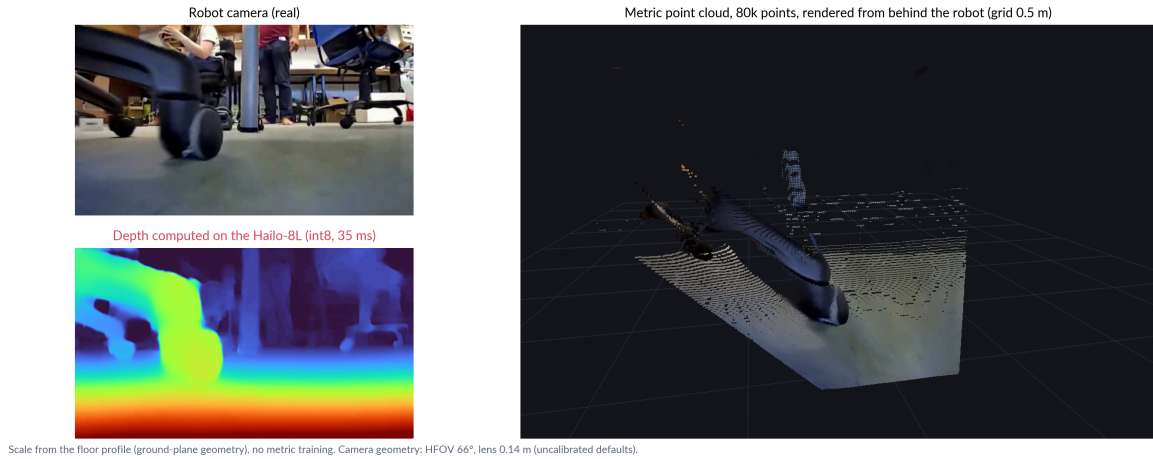


Figure 5.2. Real robot frame to metric 3D on the Pi: chip depth, scale from the floor profile, flying pixels removed. Camera geometry here uses uncalibrated defaults (66° horizontal field of view, lens 0.14 m).

5.3 Point clouds

The same floor profile is also a ruler for every pixel. Since $(f(v) - b)$ is proportional to $1/Z_{\text{floor}}(v)$, the product $K = Z_{\text{floor}}(v) (f(v) - b)$ is constant over floor rows, and any pixel's metric depth is $Z = K/(d - b)$. Back-projecting with the camera intrinsics gives a metric, coloured point cloud from one frame, with no metric training (Figure 5.2). We drop “flying pixels” at depth discontinuities (where a 3×3 neighbourhood spans more than 8 % in depth). We use the cloud for visualisation and overhang detection, not for ranging obstacles. Per pixel it is less robust than the contact scan: median error -2 to $+8$ % on obstacle pixels, but a long tail (p90 $+44$ to $+167$ % at object edges).

5.4 The map

Scan hits accumulate in a 5 cm grid of hit counts in the world frame. A cell becomes an obstacle after three hits *and* if it holds at least 30 % of the hits of the densest cell within 0.2 m. The absolute count suppresses single-frame noise: on real Hailo depth, one hit kept 53–102 phantom cells per episode and two hits 10–42; the deployed three is stricter still. The relative test stops an obstacle's sparse fringe from slowly growing it shut: a 0.9 m barrel gate otherwise read closed. Cells are never cleared. Clearing cells that later rays see through erased thin chair legs, which the Hailo's depth does not hit every frame. The world is static in our scenes. Rays also mark *seen* free space, which the planner uses (Chapter 6).

5.5 The measured limit: low bases

The remaining perception error is not in the depth model's scale but in thin, low structure. For each prop fully in view we measure how far the outermost scan hit lies *inside* the true collider edge, per side (Table 5.1).

Table 5.1. Scan side deficit (median, cm; positive = the scan sees the object narrower than its footprint) and raw near-hit phantom share, as a function of the height threshold $H_{\min}$, for ground-truth and student depth.

$H_{\min}$	GT depth, deficit	GT, phantoms	Student, deficit	Student, phantoms
5 cm (deployed)	+5.5	6.1 %	+5.1	13.7 %
3 cm	+1.7	11.5 %	+4.2	20.1 %
2 cm	−0.5	19.5 %	+2.5	24.7 %

Two facts follow. First, at the deployed threshold the ground-truth and student scans are *equally* narrow (+5.5 vs +5.1 cm). The gap is the threshold, not the network: cone base plates, sign and bin bases and shoes sit below 5 cm. Second, lowering the threshold closes the gap for ground-truth depth but not for the student, which blurs the floor-to-base boundary. Fine-tuning with the loss up-weighted below the horizon (v5) reduced phantoms (13.7→10.2 %) but not the deficit. A colour cue does close it for cones (cone deficit +5.7→+0.5 cm, no new phantoms). In the closed loop, however, the larger cones narrowed the slalom gaps and cost as much as they gained (Chapter 7).

Chapter 6

Onboard autonomy: planning, control and SLAM

Everything in this chapter runs on the Raspberry Pi 5 CPU while the Hailo-8L computes the next depth map.

6.1 Global planning: hybrid A* on sensed obstacles

Search. The planner is a hybrid A* [6] over continuous rear-axle poses (x, y, θ) with 0.1 m cells and 72 heading bins. Each expansion applies ten motion primitives, five curvatures forward and backward, each 0.25 m long. A path is collision-checked with a three-circle footprint (radius 0.16 m) against the sensed obstacles and against a *prior* drivability map of the yard derived from its reconstruction. Reversing costs twice as much as driving forward, and a change of direction costs an extra 0.6.

Heuristic. The cost-to-go is the larger of two admissible estimates: an obstacle-aware 2D geodesic distance to the goal, and a precomputed table of the obstacle-free *non-holonomic* cost of this exact lattice. The second matters: goals beside or behind the car need a reverse that holonomic heuristics cannot see. Without it, A* flooded 40,000 expansions on a recorded failure that needs one reverse.

Planning within the car's real turning ability. The simulated Traxxas turns a 0.95 m radius at full lock where the bicycle model says 0.86 m. Plans at full lock saturated the follower, and the car drifted to the outside of the turn, around the very obstacle it was curving past. Primitives are therefore planned at 75 % of lock (a 1.17 m radius), which leaves the follower steering margin.

A safety cascade. A first search uses a 0.10 m clearance margin (2 s budget). If it fails, a second relaxes it to 0.06 m and ignores the prior terrain map within 0.6 m of the start (1 s); the physics can let the car into ground the prior marks non-drivable, and it must be able to drive out. If the accepted path hugs its margin, a *wide* search at 0.20 m replaces it when at most $1.6\times$ longer (1 s). Obstacle cells the car already overlaps when planning starts (a contact) are ignored within 0.3 m of the start, so the car can back out of them, and are hard obstacles beyond it. Chapter 7 shows why that last rule matters.

Unseen space. Space no ray has observed is passable but costs three times more within 0.9 m of the start. The camera sees $\pm 34^\circ$, and a hard turn swings the body into space beside the car that it never looked at.

Asynchronous execution. Searches run in a worker process; the car holds still while a plan is computed and otherwise follows the current one. On the Pi a search takes 110 ms at the median and 547 ms at the 90th percentile (993 recorded plans, Figure 6.1).

6.2 Control and safety behaviours

A pure-pursuit follower [5] (look-ahead 0.45 m, cut to 0.35 m and throttle 0.3 where the path passes within 0.15 m of an obstacle) tracks one direction segment at a time. Short three-point-turn legs are replayed open-loop because the look-ahead exceeds them. Re-planning is triggered when:

- a newly sensed obstacle blocks the path ahead;
- the car leaves the path by more than 0.25 m;
- a segment takes far longer than it should;
- the car is *wedged*: commanded throttle but no progress for 1.5 s. A “bump” obstacle is then placed at the bumper; this is how the car learns about things it cannot see.

When no plan exists the car backs out along its own recent trail, at most three times. A retreat that makes no progress is straightened, and a pinned straight retreat is treated as a wedge. The Traxxas ESC interprets reverse after forward as a brake. Every reverse therefore passes through brake (0.30 s) and neutral (0.15 s), and the simulator models the same gate. A reactive layer scores 21 constant-curvature arcs on clearance and geodesic progress [2], as a fallback when the global planner has nothing.

6.3 SLAM

Ground-plane visual odometry. About 400 features in the floor band (up to 1.5 m ahead) are tracked frame to frame with forward–backward Lucas–Kanade [21], lifted to metric floor points with the camera height (the same geometry as the obstacle scan), and a RANSAC 2D rigid fit gives $(\Delta x, \Delta z, \Delta \theta)$. The visual-odometry pose feeds a 2D log-odds occupancy map with scan matching.

Wheel odometry and a slip guard. The car can carry a rear-axle encoder and a gyro, and the SLAM step accepts $(\Delta s, \Delta \theta)$ directly. In simulation the encoder is the rear-wheel joint velocity times the wheel radius, and the gyro is the true yaw rate plus a $0.1^\circ/\text{s}$ bias and $0.3^\circ/\sqrt{s}$ noise. Wheels slip when the car pushes against something, so an encoder alone ran up to 5 m off. The guard declares the car *still* when the rotation-compensated median floor flow is below 2 px, and only when the encoder claims at least 2 cm per frame. Without that second condition, a slow legitimate reverse (≈ 1 px of floor flow) was zeroed and reverse odometry read 7 % short. With it, the shortfall is 3 %.

Table 6.1. Self-localisation alternatives, offline on 50 recorded 35-second episodes (absolute trajectory error).

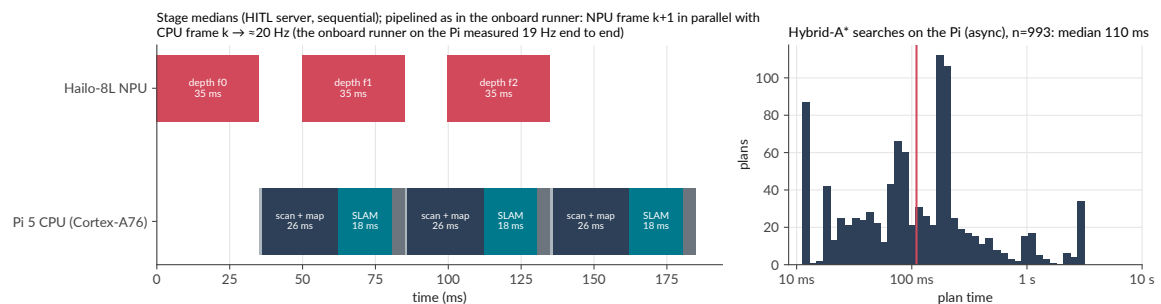
Odometry source	ATE median (m)	ATE p90 (m)
Ground-plane VO (camera only)	0.42	1.32
VO translation + gyro heading (an IMU alone)	0.36	1.04
Encoder + VO heading	0.22	1.18
Encoder + Ackermann heading from the steering command	0.25	0.61
Encoder + gyro	0.08	0.19

Table 6.1 carries the hardware recommendation for the real car: an IMU alone barely helps, and the *wheel encoder* is what makes own-pose navigation work. In the closed loop, with encoder and gyro, the onboard estimate stays within 0.13 m of the truth at the median and 0.99 m at the 90th percentile over ten seeds (Figure 7.4).

6.4 The real-time budget on the Pi

Table 6.2. Measured on the Raspberry Pi 5 + Hailo-8L as the robot computer of the hardware-in-the-loop suite (38,207 frames; A*: 993 plans).

Stage (Raspberry Pi 5 + Hailo-8L)	median (ms)	p90 (ms)	samples
JPEG decode	1.0	1.6	38,207
Depth network on the NPU (288×512, int8)	35.1	35.8	38,207
CPU: scan + map + SLAM + navigator step	48.9	54.7	38,207
of which SLAM (own-pose runs)	18.5	23.2	20,183
of which navigator step	4.2	6.2	37,858
Hybrid-A* search (asynchronous worker)	109.9	546.8	993
Round trip, Pi ↔ simulator (incl. network)	88.0	93.9	38,207

Real-time budget on the Raspberry Pi 5 + Hailo-8L (measured on the robot computer, 38k frames)**Figure 6.1.** Left: per-frame stages (medians) and how the onboard runner overlaps them. Right: hybrid-A* search times on the Pi.

The depth network occupies the NPU for 35 ms per frame and the CPU stages take about 49 ms, so with the NPU working on frame $k+1$ while the CPU handles frame k the loop runs at about 20 Hz on simulator-sized frames. On real 1280×720 camera video the standalone onboard runner, with the deployed code, processes 300 frames in 18.1 s with ground-VO

SLAM (16.5 Hz; feature detection and tracking at full resolution take 19 ms of it) and in 11.1 s without it (27 Hz, limited by the NPU). In the hardware-in-the-loop suite the round trip to the simulator adds the network link: 88 ms at the median. With the final deployed code the real board measured 35.0 ms on the NPU, 46.7 ms of CPU at the median (64.1 ms at the 90th percentile) and an 85 ms round trip, over 9,371 replies. The simulator applies each command only when it arrives, so this latency is part of every result in Chapter 7.

Sustained operation. Real time must also hold for hours, not only at the median. Between its last boot and the end of the real-board arms the Pi served 1000 hardware-in-the-loop episodes over 28 h of wall time (380,127 logged control steps). NPU time did not drift: 35.1 ms at the median and 35.9 ms at the 99th percentile. Afterwards, the firmware's sticky throttle register still read 0x0: since boot there had been no under-voltage, no frequency capping and no thermal throttling.

Chapter 7

Hardware-in-the-loop evaluation

7.1 Set-up

World. NVIDIA Isaac Sim with Isaac Lab [25]. The yard is a Gaussian-splat reconstruction [16, 40] of a real yard, with a heightfield collider for its terrain. The robot is a model of the project’s Traxxas with an Ackermann PWM action: ± 0.349 rad steering and up to 0.72 m/s. The camera is level, 504×280 with a 69.4° horizontal field of view, 0.13 m above the ground.

Obstacles. Eleven prop types from public asset libraries are placed on the rendered ground: an office chair, a cantilever chair, a school chair, a traffic cone, a cardboard box, a barrel, a trash can, a wet-floor sign, a crate and two posed people. Each has an invisible kinematic collider sized to what the car can actually hit:

- round bodies use their body radius;
- boxy props use their box;
- legged chairs use only the cells where their surface crosses the car’s body band (4–30 cm). A solid cylinder had made an “invisible fence” between chair legs;
- people use a 0.25 m cylinder around feet and legs.

Scenes. Ten frozen layouts with 3–6 m between start and goal: chair, box offset, cone slalom, barrel gate, chair cluster, person on path, mixed clutter, crate wall with a gap, person and cones, and barrel. Scenes 07–09 were never seen by the depth network in training.

Protocol. Each configuration (an *arm*) runs all ten scenes for at least five seeds; a seed jitters the start by up to 0.15 m. An episode ends on success (car within 0.45 m of the goal), after 35 s, or if the car tips. It is *clean* if no point of the car’s real 0.557×0.294 m footprint touched any collider at any time. Every frame is JPEG-encoded and sent to the robot computer, and the returned command is applied only when it arrives.



Figure 7.1. The deployed stack with the real Pi + Hailo-8L in the loop, on its own SLAM pose. Each panel shows the chase camera (top left), the robot's camera (top right), the depth computed on the Hailo-8L with obstacle pixels outlined and the commanded arc (bottom left), and the bird's-eye view with the SLAM path and the current plan (bottom right). Left: cone slalom, 6 s in. Right: people and cones, backing away from a cone seen at close range (this episode reached the goal with a brush).

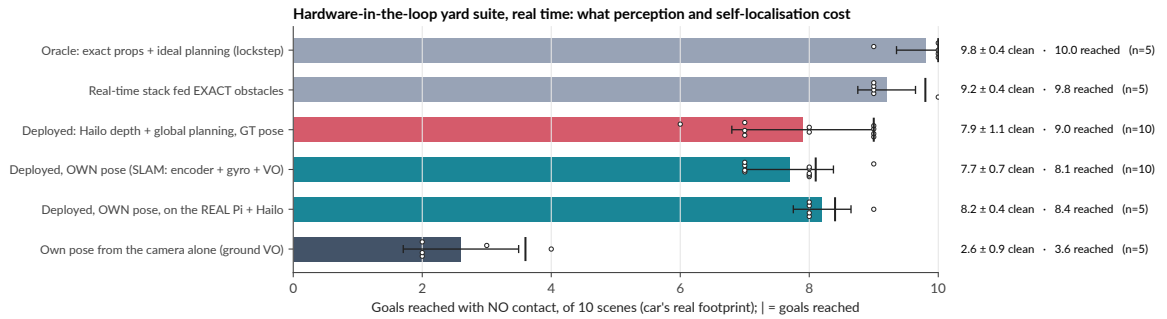


Figure 7.2. The error budget. Bars: clean goals of 10 (mean ± sd over seeds, dots = seeds); ticks: goals reached.

A faithful local brain. The real Pi and Hailo-8L ran the first arms. To run arms in parallel, the identical Pi code also runs on the laptop. The student executes in PyTorch with outputs identical to the chip (Pearson 1.0000 to its ONNX twin, which equals the HEF). Replies are padded to the Pi's measured round trip (84 ms median), and asynchronous plans are stretched by $2.35\times$, the measured Pi-to-laptop planner speed ratio. Replicating a Pi arm locally gave 6.6 vs 7.2 clean and 5.6 vs 5.6 on the stricter circle metric, within seed noise. The final, deployed configuration was then run again with the real board in the loop (own pose, seeds 0–4): 8.2 ± 0.4 clean against 7.6 for the local brain on the same seeds. The real round trip was 85 ms at the median (90th percentile 103 ms, 99th 113 ms, 9,371 replies), against the local brain's 84 ms. The local brain is, if anything, conservative.

7.2 Results

The error budget (Figure 7.2).

- An **oracle** that knows every prop exactly and plans in lockstep reaches **9.8** clean of 10: a perfect score is feasible on this suite.
- The same **real-time** stack fed **exact obstacles** reaches **9.2**. Latency, the follower and the ESC gate cost about 0.6.

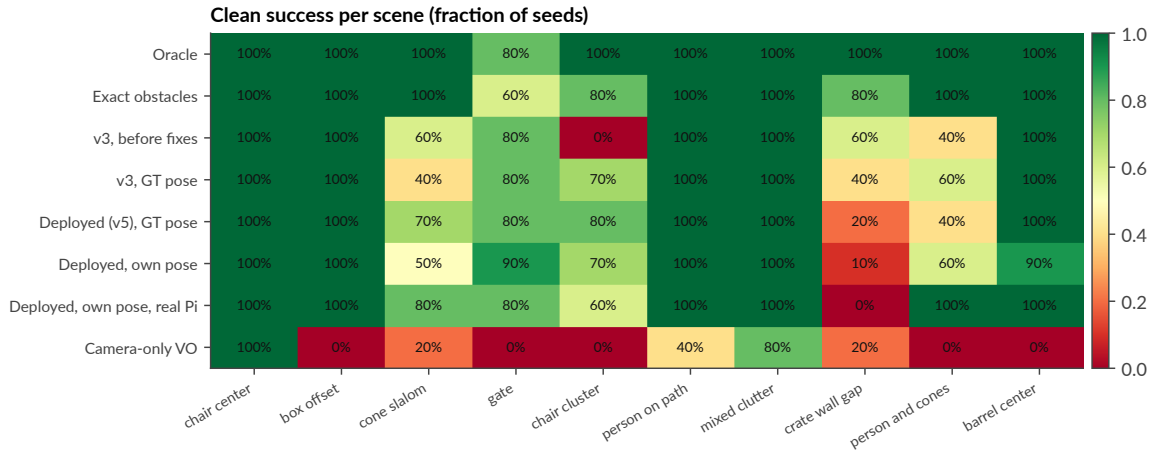


Figure 7.3. Clean success per scene. Remaining failures concentrate on the cone slalom, the gate, the chair cluster, the crate wall and the person-and-cones scene.

- With **depth from the Hailo-8L**, the deployed stack reaches 7.9 ± 1.1 (9.0 reached) over ten seeds. Perception costs about 1.3.
- On its **own pose** (SLAM with simulated encoder and gyro) it reaches 7.7 ± 0.7 (8.1 reached) over ten seeds, and 8.2 ± 0.4 with the real Pi and Hailo-8L in the loop (five seeds). Self-localisation costs little in clean goals (0.2) but 0.9 in goals reached. The failure mode changes with it: with simulator pose most misses reach the goal after a graze (11 of 21), while on own pose most are timeouts (18 of 23), usually after a brush, as drift misplaces the goal or mapped obstacles.
- With **camera-only odometry**, 2.6. A wheel encoder is not optional.

Student v3 or v5? The floor-weighted student v5 replaced v3 on the strength of ten seeds per arm, and the evidence is modest, so we state it exactly. With simulator pose the two tie (7.9 ± 1.0 vs 7.9 ± 1.1), with per-scene differences in both directions: v5 is better on the cone slalom (7 vs 4 of 10) and worse on the crate wall and the person-and-cones scene (2 vs 4, 4 vs 6). On the robot's own pose v5 is ahead, 7.7 ± 0.7 against 7.1 ± 0.9 (8.1 vs 7.4 reached), mainly through the chair cluster (7 vs 4) and person-and-cones (6 vs 4). The difference is not significant (permutation test $p \approx 0.17$). v5 also costs nothing: same architecture, same speed on the chip (27.1 FPS), int8 equal to float at Pearson 0.99993 (p5 0.99569), and the lower AbsRel on obstacle pixels of the two (0.197 vs 0.204). It is deployed because it is never measurably worse and is ahead on the pose the real car will use.

7.3 Failure analysis

Thin chair frames. Under the cantilever chair's seat the floor model took the seat for floor (Chapter 5). The horizon-anchored fix, together with the planner fixes below, took the chair cluster from 0/5 to 3/5 clean.

Table 7.1. Every evaluated arm (clean and reached goals of 10, mean $\pm$ sd over n seeds, scored on the car’s real footprint).

Arm	Configuration	n	Clean (of 10)	Reached
<i>Ceilings (no perception error)</i>				
o13	Oracle: exact props, the Navigator’s planner (lockstep)	5	9.8 ± 0.4	10.0 ± 0.0
l21x	Real-time stack, EXACT obstacles (no perception)	5	9.2 ± 0.4	9.8 ± 0.4
<i>Hailo depth, simulator (GT) pose</i>				
p12rt	Student v1 + global planner (Pi)	5	7.2 ± 1.3	7.8 ± 1.1
p15rt	Student v3 + global planner (Pi)	5	7.2 ± 1.1	9.0 ± 1.0
l22rt	v3 + PLAN_STEER 0.85 + straight-back retreat	5	7.4 ± 0.5	9.2 ± 0.4
l26rt	+ floor-from-horizon, pinned-retreat wedge, blocked fix	5	7.2 ± 1.3	8.8 ± 1.1
l30rt	+ escape-radius planning	5	6.8 ± 1.1	8.6 ± 1.1
l31rt	+ PLAN_STEER 0.75 + floating overhangs (the deployed planner, v3)	10	7.9 ± 1.0	9.0 ± 0.8
l32rt	+ base skirt on small obstacles (rejected)	5	7.2 ± 1.1	8.2 ± 0.8
l35rt	+ look first (neutral on own pose; not deployed)	5	8.0 ± 0.0	8.8 ± 0.8
l36rt	+ terrain-escape pass (neutral)	5	7.8 ± 1.1	9.2 ± 0.4
l37rt	+ faster follower (neutral)	5	7.4 ± 0.5	8.6 ± 0.9
l38rt	Student v5 (floor-weighted loss) [deployed]	10	7.9 ± 1.1	9.0 ± 0.9
l39rt	+ cone colour contacts (rejected)	5	7.8 ± 1.1	8.6 ± 1.1
<i>Hailo depth, the robot’s OWN pose</i>				
p12full	Own pose: ground-plane VO only	5	2.6 ± 0.9	3.6 ± 0.5
p16imu	Own pose: v3 + sim encoder + gyro	5	6.0 ± 1.4	7.6 ± 1.1
l22imu	Own pose: l22 planner + encoder + gyro	5	6.2 ± 1.3	7.4 ± 1.3
l31imu	Own pose: deployed planner + encoder + gyro	5	7.0 ± 0.7	7.0 ± 0.7
l34imu	Own pose: + encoder-gated slip guard (deployed planner, v3)	10	7.1 ± 0.9	7.4 ± 1.0
l35imu	Own pose: + look first	10	6.8 ± 0.9	7.2 ± 0.8
l37imu	Own pose: + faster follower	5	7.2 ± 0.8	7.8 ± 1.1
l38imu	Own pose: student v5 [deployed]	10	7.7 ± 0.7	8.1 ± 0.7
p38imu	Own pose: student v5 on the REAL Pi + Hailo [deployed]	5	8.2 ± 0.4	8.4 ± 0.5
l39imu	Own pose: + cone colour	5	6.4 ± 0.5	7.2 ± 0.8

Onboard SLAM + global path finding in the real-time loop (own pose: simulated wheel encoder + gyro + ground VO)

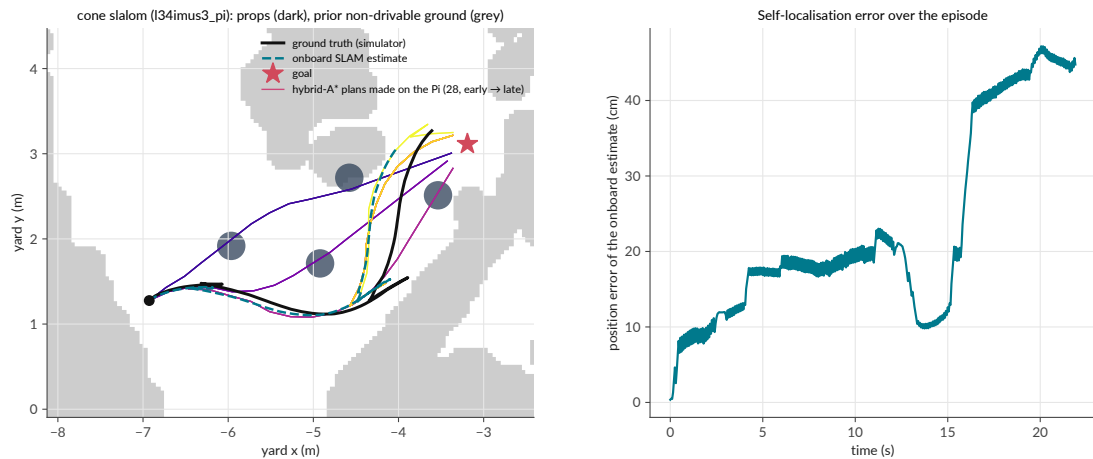


Figure 7.4. Onboard SLAM and global planning on the robot's own pose: every hybrid-A* plan the Pi made in one slalom episode, the SLAM estimate against the truth, and the localisation error over time. The car reached the goal despite 0.47 m peak drift.

Obstacles the camera has not seen. In the crate-wall scene the goal starts about 63° off the camera axis, and one crate lies $65\text{--}74^\circ$ to the left, outside the $\pm 34^\circ$ view. It is never mapped while the car plans, so the first plans run through it. The car approaches, sees it, blocks, reverses and repeats. Penalising unseen space more widely did not help: every route to the goal crosses unseen space, and searches got $6\text{--}20\times$ slower. A “look first” arc toward the goal raised the GT-pose score to 8.0 ± 0.0 but was neutral on own pose at ten seeds, so it is not deployed. This is an exploration problem, not a perception one.

Sides of small objects. The remaining contacts are almost all the car's front corner grazing the side of a cone, a person or a crate, which the scan places 5–15 cm too far out. This is the low-base limit of Section 5.5. Uniform margin increases were null because they also close the narrow gaps that other scenes require.

A regression we caused and fixed. One planner fix made the “path is blocked” check consistent with the planner, which ignored obstacle cells already overlapping the car. That removed an accidental safety net: the old, inconsistent check had been vetoing plans that ran through a crate face the bumper was touching. The crate-wall scene fell from 3/5 to 1/5. The proper fix ignores overlapped cells only within an escape radius of 0.3 m and treats them as obstacles beyond it, in both the planner and the check. A unit test reproduces the trap: the old code planned straight through a touched crate face, and the new code goes around it.

7.4 What did not help

The last row of Table 7.2 is worth stating plainly: **the student drives better than the simulator's own depth buffer.** The renderer's depth for a splat scene carries speckle and floaters near the floor (visible in Figure 4.3). The student, trained to the teacher's clean shape and

Table 7.2. Interventions that did not improve clean success at ≥ 5 seeds (clean goals of 10, against the baseline of the time on the same seeds, e.g. 7.6 for the v3 planner on seeds 0–4).

Intervention	Outcome	Clean
Base skirt: +6 cm on small mapped clusters	04 improves to 5/5 but the slalom closes	7.2 vs 7.6
Cone-colour contacts (orange mask)	cone gap closed offline; slalom timeouts in the loop	7.8 vs 7.6; own pose 6.4 vs 7.1
“Look first” arc toward an off-axis goal	neutral on own pose (10 seeds)	GT 8.0; own 6.8 vs 7.1
Third planning pass, terrain free within 1.5 m	fixes targeted timeouts, contacts move	7.8 vs 8.0
Faster follower (0.6 / 0.45 throttle)	own-pose drift p90 0.99→1.63 m	7.4 vs 7.6
Uniform blocked-path margin 0.04 m	extra re-plans stop the car	6.6 vs 7.4
Mapping low hits beside tall ones	—	7.0 vs 7.2
2× supersampled camera pixels	—	own pose 2.6 vs 2.6
Latency-compensated follower	worse on a synthetic S-curve; reverted	—
Renderer GT depth instead of the student	GT depth has speckle and floaters (GT arm ran lockstep, so not a matched pair)	5.2 vs 6.8

then to that ground truth, produces a cleaner signal for ground-contact ranging. The comparison is conservative in the renderer’s favour: its arm ran in lockstep, without the round-trip latency the student’s arm had to absorb.

7.5 Statistics

Seed-to-seed standard deviations are about one goal in ten, so differences below about one goal at $n = 5$ are not resolvable. We report means, standard deviations and every seed, and extend arms to ten seeds when a decision depends on them. Contacts are scored on the car’s real rectangle rather than the simulator’s 0.22 m contact circle, which over-reports side brushes. A small number of seeded starts are thrown by the physics settle into contact with a prop; excluding contacts already present before the first command moves the v3 GT-pose score from 7.9 to 8.0 and leaves both deployed v5 arms unchanged. We report the stricter number.

Chapter 8

Discussion

8.1 What is new

A depth network the Hailo-8L can actually run. The common assumption is that a foundation model reaches a small NPU through better quantisation. On the Hailo-8L it does not. Attention in int8 is structurally lossy at a few hundred tokens, and neither our build nor the vendor’s survives it (Section 3.2). Distilling into a purely convolutional student turns this from a quantisation problem into a training problem, and the result is unusually clean. The int8 student equals its float twin at $r = 0.99995$, so quantisation cost disappears from the error budget entirely. The student is also the most accurate depth network on the chip by a margin that no speed trade explains (Table 4.1). The design choices that make it work are simple: an ImageNet ResNet-34 encoder, a U-Net decoder, a scale-invariant loss, and a sim-ground-truth fine-tune. We see that simplicity as a strength, because it transfers to any integer-only accelerator.

Metric navigation without metric depth. Most depth-based navigation systems either learn metric depth or fuse relative depth with external scale. Ground-contact ranging needs neither. It asks the network only where each column’s floor ends, and the camera height does the rest. It is robust to the frame-to-frame scale drift of relative depth, which is the property that matters most on a moving robot. The failure modes it has are geometric and nameable: overhangs, bases lower than the threshold, and surfaces filling a whole row. Each has a measured detector or fix in Chapter 5.

A complete stack in a few watts. Depth (35 ms on the NPU), a 69° metric scan, a hit-count map, global hybrid-A* planning with a safety cascade, pure-pursuit control and a SLAM front end all fit on a Raspberry Pi 5 at 16–27 Hz. Finke et al. [8] do comparable perception on a Jetson AGX Orin, a computer an order of magnitude more expensive; MonoNav [34] computes depth off-board.

An evaluation that explains itself. Oracle, exact-obstacle, GT-pose and own-pose arms decompose the gap to a perfect score into execution (0.6), perception (1.3) and localisation (0.2 in clean goals, 0.9 in goals reached). Without that decomposition, the many interventions of Table 7.2 would have been tuned blindly.

8.2 Limits, stated plainly

- **Not 10/10.** The deployed system reaches 7.9 of 10 clean with simulator pose and 7.7 on its own pose. The oracle reaches 9.8, so the gap is real and not a property of the benchmark.
- **Low bases.** Cone plates, sign bases and shoes below 5 cm are invisible to the scan. Ground-truth depth would resolve them at a 2 cm threshold; the student's depth does not (Table 5.1).
- **Exploration.** With a $\pm 34^\circ$ camera the car commits to plans through space it has not seen. One scene (crate wall) depends on this more than on anything else.
- **Own pose uses simulated sensors.** The encoder and gyro of Table 6.1 are simulated; the real car has neither yet. Camera-only odometry reaches only 2.6 of 10.
- **One environment.** The simulated yard is a reconstruction of one real yard, and the student was fine-tuned there (on held-out scenes). Real-world validity rests on real-video agreement with the teacher ($r = 0.995$), not on real metric ground truth.
- **No autonomous real drive yet.** The real car has driven tethered under teleoperation. Autonomous drives are gated on a human confirming three runaway failsafes in session. The Pi's camera was not detected when this document was prepared.

8.3 Threats to validity

- *Simulator depth as ground truth.* The renderer's depth for a splat scene has speckle and floaters near the floor. We mitigate this with the obstacle-pixel region (Chapter 4), but absolute AbsRel values should be read as relative comparisons.
- *Seed noise.* About one goal in ten. We use five seeds, and ten where a decision depends on it. Interventions that differ by less than about one goal are reported as neutral.
- *Local brain.* Most later arms ran the Pi's code on the laptop with the Pi's measured latency and planning speed. A replicated Pi arm matched within noise, but small timing effects cannot be ruled out.
- *Metric choice.* Scoring on the car's real footprint is stricter than the simulator's contact circle in some places (front corners) and more lenient in others (sides). We report the footprint metric throughout.

Chapter 9

Conclusion and future work

A foundation-quality monocular depth network can run on a Raspberry Pi 5 with a Hailo-8L, but not as a transformer. Distilled into an attention-free student, it runs at 27 FPS at 288×512 , its int8 output equals its float output, and it is the most accurate depth network on the chip. Ranged by ground-contact geometry, its relative depth becomes a metric scan without metric training. On that scan the Pi itself maps, plans with hybrid A*, follows, and localises, at 16–27 Hz. With the real board in a real-time loop through a reconstructed yard, the system reaches the goal without contact in 7.9 of 10 scenes on simulator pose and 7.7 on its own pose (8.2 with the real board in the loop), where the same stack with perfect obstacles reaches 9.2.

9.1 Closing the gap

The measurements point at three concrete next steps, in order of expected value:

1. **Exploration that looks before it commits.** Treat unseen space near a plan as uncertain and steer the camera toward it (perception-aware heading, as in MonoSpheres [24]), instead of planning through it.
2. **Low-structure perception.** Supervise the floor-to-base boundary directly. That could mean a contact-height head trained on rendered ground truth, or semantic detection of cones and people on the same NPU, used conservatively so it does not close narrow gaps.
3. **Temporal depth.** Thin structures flicker from frame to frame. A recurrent or memory-based student, in the spirit of AsyncMDE [22] and Video Depth Anything [4], could stabilise what single frames miss.

9.2 Before the first autonomous drive

1. Fit a rear-axle wheel encoder and a gyro (Table 6.1: 0.08 m median ATE against 0.42 for the camera alone).
2. Calibrate the camera's field of view, height and *pitch*: a 3° downward tilt moves the horizon by about 50 pixels at 720p and changes every range.

3. Check the student's ranges on real ground at tape-measured 0.5, 1.0 and 1.5 m.
4. Do a dry run with the wheels up, then tethered armed runs at 0.25 throttle with the operator deadman in hand, and only after a human has confirmed the three runaway failsafes live in session.

Bibliography

- [1] Alexey Bochkovskiy, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R. Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. In *International Conference on Learning Representations (ICLR)*, 2025.
- [2] Johann Borenstein and Yoram Koren. The vector field histogram – fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation*, 7(3):278–288, 1991.
- [3] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.
- [4] Sili Chen, Hengkai Guo, Shengnan Zhu, Feihu Zhang, Zilong Huang, Jiashi Feng, and Bingyi Kang. Video depth anything: Consistent depth estimation for super-long videos. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [5] R. Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical Report CMU-RI-TR-92-01, Carnegie Mellon University, Robotics Institute, 1992.
- [6] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Path planning for autonomous vehicles in unknown semi-structured environments. *The International Journal of Robotics Research*, 29(5):485–501, 2010.
- [7] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [8] Jan Finke, Wayne Paul Martis, Adrian Schmelter, Lars Erbach, Christian Jestel, and Marvin Wiedemann. Learning vision-based omnidirectional navigation: A teacher-student approach using monocular depth estimation. *arXiv preprint arXiv:2603.01999*, 2026.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [10] Xiankang He, Dongyan Guo, Hongji Li, RuiBo Li, Ying Cui, and Chi Zhang. Distill any depth: Distillation creates a stronger monocular depth estimator. *arXiv preprint arXiv:2502.19204*, 2025.
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [12] Mu Hu, Wei Yin, et al. Metric3d v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

- [13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [14] M. Jalalnezhad, B. T. Sayed, Y. Alotaibi, et al. Real-time vision-based obstacle avoidance for mobile robots using lightweight monocular depth estimation and behavior-driven control. *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, 47:668, 2025.
- [15] Bingxin Ke, Anton Obukhov, Shengyu Huang, Nando Metzger, Rodrigo Caye Daudt, and Konrad Schindler. Repurposing diffusion-based image generators for monocular depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [16] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.
- [17] Zhikai Li, Junrui Xiao, Lianwei Yang, and Qingyi Gu. Repq-vit: Scale reparameterization for post-training quantization of vision transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [18] Haotong Lin, Sili Chen, Jun Hao Liew, et al. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [19] Yang Lin, Tianyu Zhang, Peiqin Sun, Zheng Li, and Shuchang Zhou. Fq-vit: Post-training quantization for fully quantized vision transformer. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.
- [20] Muxin Liu, Xiaoyang Lyu, Yang-Tian Sun, Yi-Hua Huang, Ziyi Yang, Peng Dai, and Xiaojuan Qi. Monocular depth estimation from a single image: Progress and opportunities. *arXiv preprint arXiv:2609.01172 (accepted to Computational Visual Media)*, 2026.
- [21] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 1981.
- [22] Lianjie Ma, Yuquan Li, Bingzheng Jiang, Ziming Zhong, Han Ding, and Lijun Zhu. Asyncmde: Real-time monocular depth estimation via asynchronous spatial memory. *arXiv preprint arXiv:2603.10438*, 2026.
- [23] Riku Murai, Eric Dexheimer, and Andrew J. Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [24] Tomáš Musil, Matěj Petrлік, and Martin Saska. Monospheres: Large-scale monocular slam-based uav exploration through perception-coupled mapping and planning. *IEEE Robotics and Automation Letters (arXiv:2511.17299)*, 2025.
- [25] NVIDIA et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.
- [26] Luigi Piccinelli, Yung-Hsu Yang, Christos Sakaridis, Mattia Segu, Siyuan Li, Luc Van Gool, and Fisher Yu. Unidepth: Universal monocular metric depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [27] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [28] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1623–1637, 2022.
- [29] Raspberry Pi Ltd. Raspberry pi ai kit: product brief. <https://datasheets.raspberrypi.com/ai-kit/raspberry-pi-ai-kit-product-brief.pdf>, 2024. Hailo-8L M.2 module, 13 TOPS, published June 2024.
- [30] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.
- [31] Pascal Roth, Julian Nubert, Fan Yang, Mayank Mittal, and Marco Hutter. Viplanner: Visual semantic imperative learning for local navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [32] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. Vint: A foundation model for visual navigation. In *Conference on Robot Learning (CoRL)*, 2023.
- [33] Basant Sharma, Prajyot Jadhav, Pranjal Paul, K. Madhava Krishna, and Arun Kumar Singh. Monomp: Monocular vision based navigation with learned collision model and risk-aware model predictive control. *IEEE Robotics and Automation Letters* (*arXiv:2508.07387*), 2026.
- [34] Nathaniel Simon and Anirudha Majumdar. Mononav: Mav navigation via monocular depth estimation and reconstruction. In *International Symposium on Experimental Robotics (ISER)*, 2023.
- [35] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [36] Libo Sun, Jia-Wang Bian, et al. Sc-depthv3: Robust self-supervised monocular depth estimation for dynamic scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(1):497–508, 2024.
- [37] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [38] Zachary Teed, Lahav Lipson, and Jia Deng. Deep patch visual odometry. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [39] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. Fast-depth: Fast monocular depth estimation on embedded systems. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [40] Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moenne-Loccoz, and Zan Gojcic. 3dgut: Enabling distorted cameras and secondary rays in gaussian splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [41] Fan Yang, Chen Wang, Cesar Cadena, and Marco Hutter. iplanner: Imperative path planning. In *Robotics: Science and Systems (RSS)*, 2023.

- [42] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [43] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [44] Zhihang Yuan, Chenhao Xue, Yiqi Chen, Qiang Wu, and Guangyu Sun. Ptg4vit: Post-training quantization for vision transformers with twin uniform quantization. In *European Conference on Computer Vision (ECCV)*, 2022.
- [45] Ning Zhang, Francesco Nex, George Vosselman, and Norman Kerle. Lite-mono: A lightweight cnn and transformer architecture for self-supervised monocular depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Appendix A

Reproduction

All code is in the project repository (itaim18/DANIEL). Paths below are relative to its root.

A.1 Depth student

<code>board/pi5/student_depth.py</code>	frames / label / train / eval / export; --floor-w for v5
<code>board/pi5/student_hailo.all</code>	Hailo compile recipe (normalisation, optimisation level 1)
<code>board/pi5/edge_depth.py</code>	on-device inference for every HEF (centre crop at the model's aspect)
<code>experiments/2026-09-30_student_depth/</code>	weights, HEFs, training logs, README

A.2 Benchmark (Chapter 4)

All in `experiments/2026-10-02_thesis_bench/`:

<code>frames.json</code>	the 400 held-out benchmark frames
<code>bench_baselines.py</code>	GPU predictions: Depth Anything V2 S/B/L and Depth Anything 3
<code>bench_students.py</code>	GPU predictions of the students (the chip's float twin)
<code>chipbench.py</code>	on the Pi: every HEF on the same frames, outputs + timing
<code>score.py</code>	alignment, regions, metrics → <code>scores.json</code>
<code>fig_*.py, make_demo.py</code>	every figure of this thesis and the 1080p demo video
<code>gpu_timing.py, cpubench.py</code>	float-GPU and Pi-CPU latency references
<code>hitl_data.py, make_reel.py</code>	per-seed loop results; labelled reels of a run
<code>report/build.py</code>	the web report, from the same JSON files

Tables: `thesis/make_tables.py` generates every table from the JSON files (no hand-copied numbers).

A.3 Navigation stack and HITL

<code>board/pi5/avoid.py</code>	ground-contact scan, floor model, detectors, reactive layer (self-tested)
<code>board/pi5/pathplan.py</code>	map, hybrid A*, cascade, follower, Navigator (self-tested)
<code>board/pi5/slam2d.py</code>	ground VO, SLAM, wheel odometry, slip guard (self-tested)
<code>board/pi5/hitl_pi.py</code>	the robot computer's server (Hailo or local torch backend)
<code>board/pi5/avoid_drive.py</code>	the onboard runner for the real car (dry-run by default)
<code>wheeled_lab/rl/hitl_sim.py</code>	the Isaac Sim side of the loop
<code>experiments/2026-09-29_hitl/</code>	scenarios, per-episode logs, summaries
<code>docs/findings.md</code>	the dated record of every finding, including retractions

A.4 Commands

```
# on the Pi: serve depth + planning for the simulator
python3 hitl_pi.py          # default HEF: ~/models/dav2student_v5_288x512.hef
# on the workstation: one arm, real time, own pose with wheel odometry
./run_headless.sh log.txt hitl_sim.py --brain pi --nav --slam --realtime \
  --esc-gate --cam-h 0.133 --stuck-s 35 --odo imu --seed 1
# benchmark + figures + thesis
python3 score.py && python3 fig_bench.py && cd thesis && tectonic main.tex
```